Model Training_Conversion

Model Training_Conversion
1、Model Training
2、Model Conversion
References

After completing the dataset annotation tutorial content, we can use the board to start training the model.

```
This tutorial only introduces CLI model training and conversion, you can refer to
the official website for Python case modifications.
```

1、Model Training

Use CLI commands to directly train the model

```
cd /home/jetson/ultralytics/ultralytics/data/yahboom_data/orange_data
```

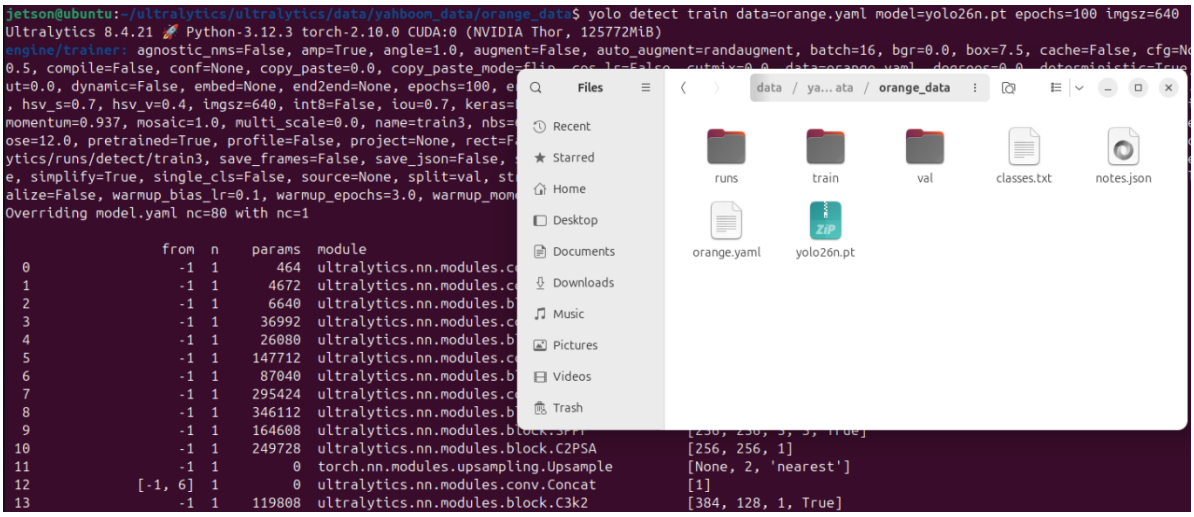
```
yolo detect train data=orange.yaml model=yolo26n.pt epochs=100 imgsz=640
```

`data`: Dataset configuration file

`model`: Pre-trained model file

`epochs`: Number of training rounds

`imgsz`: Specify input image size



```
Transferred 606/708 items from pretrained weights
AMP: running Automatic Mixed Precision (AMP) checks...
AMP: checks passed ✅
train: Fast image access ✅ (ping: 0.0±0.0 ms, read: 4887.5±3396.1 MB/s, size: 508.2 KB)
train: Scanning /home/jetson/ultralytics/ultralytics/data/yahboom_data/orange_data/train/labels.cache... 153 images, 0 backgrou
val: Fast image access ✅ (ping: 0.0±0.0 ms, read: 3480.0±4132.4 MB/s, size: 409.7 KB)
val: Scanning /home/jetson/ultralytics/ultralytics/data/yahboom_data/orange_data/val/labels.cache... 20 images, 0 backgrounds,
optimizer: 'optimizer=auto' found, ignoring 'lr0=0.01' and 'momentum=0.937' and determining best 'optimizer', 'lr0' and 'moment
optimizer: AdamW(lr=0.002, momentum=0.9) with parameter groups 114 weight(decay=0.0), 126 weight(decay=0.0005), 126 bias(decay=
Plotting labels to /home/jetson/ultralytics/runs/detect/train/labels.jpg...
Image sizes 640 train, 640 val
Using 8 dataloader workers
Logging results to /home/jetson/ultralytics/runs/detect/train
Starting training for 100 epochs...

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      1/100      2.56G     0.5452      6.288   0.006855         19        640: 100% ━━━━━━━━━━━━ 10/10 1.2it/s 8.4s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 1/1 1.1it/s 0.9s
                   all         20         20    0.00333          1      0.574      0.458

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      2/100      2.56G     0.5048      5.176   0.005876         20        640: 100% ━━━━━━━━━━━━ 10/10 4.9it/s 2.0s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 1/1 11.9it/s 0.1s
                   all         20         20    0.00333          1      0.694      0.618

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      3/100      2.56G     0.5746      4.794   0.006855         18        640: 100% ━━━━━━━━━━━━ 10/10 5.1it/s 2.0s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 1/1 11.2it/s 0.1s
                   all         20         20    0.00333          1      0.674      0.629
```

```
      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
     99/100      2.58G     0.2471      0.249   0.003648          9        640: 100% ━━━━━━━━━━━━ 10/10 5.5it/s 1.8s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 1/1 12.2it/s 0.1s
                   all         20         20      0.992          1      0.995      0.952

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
    100/100      2.58G     0.2514     0.2596   0.003862          9        640: 100% ━━━━━━━━━━━━ 10/10 5.4it/s 1.8s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 1/1 11.2it/s 0.1s
                   all         20         20      0.996          1      0.995      0.948

100 epochs completed in 0.065 hours.
Optimizer stripped from /home/jetson/ultralytics/runs/detect/train/weights/last.pt, 5.4MB
Optimizer stripped from /home/jetson/ultralytics/runs/detect/train/weights/best.pt, 5.4MB

Validating /home/jetson/ultralytics/runs/detect/train/weights/best.pt...
Ultralytics 8.4.21 🚀 Python-3.12.3 torch-2.10.0 CUDA:0 (NVIDIA Thor, 125772MiB)
YOLO26n summary (fused): 122 layers, 2,375,031 parameters, 0 gradients, 5.2 GFLOPs
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 1/1 14.5it/s 0.1s
                   all         20         20      0.997          1      0.995      0.966
Speed: 0.1ms preprocess, 1.5ms inference, 0.0ms loss, 0.1ms postprocess per image
Results saved to /home/jetson/ultralytics/runs/detect/train
💡 Learn more at https://docs.ultralytics.com/modes/train
jetson@ubuntu:~/ultralytics/ultralytics/data/yahboom_data/orange_data$
```

Finally, the trained model-related files are stored in the /home/jetson/ultralytics/ultralytics/data/yahboom_data/orange_data/runs/detect/train folder.

Note: Each time training is run, the number after train will automatically increment by 1, please record this path, and use this new model path later.

2、Model Conversion

The final model will be generated in the runs folder: generally select the best.pt file for use

```
/home/jetson/ultralytics/runs/detect/train/weights
```

Convert PyTorch format model to TensorRT:

```
cd /home/jetson/ultralytics/runs/detect/train/weights
ls
```

```
yolo export model=best.pt format=engine
```

```
jetson@ubuntu:~/ultralytics/ultralytics$ cd /home/jetson/ultralytics/runs/detect
/train/weights
jetson@ubuntu:~/ultralytics/runs/detect/train/weights$ ls
best.pt  last.pt
jetson@ubuntu:~/ultralytics/runs/detect/train/weights$ yolo export model=best.pt
 format=engine
WARNING ⚠️TensorRT requires GPU export, automatically assigning device=0
Ultralytics 8.4.21 🚀 Python-3.12.3 torch-2.10.0 CUDA:0 (NVIDIA Thor, 125772MiB)
YOLO26n summary (fused): 122 layers, 2,375,031 parameters, 0 gradients, 5.2 GFLO
Ps

PyTorch: starting from 'best.pt' with input shape (1, 3, 640, 640) BCHW and outp
ut shape(s) (1, 300, 6) (5.1 MB)
```

Wait for the model conversion to complete, best.engine and best.onnx files will be generated in the current directory.

```
ls
```

```
[03/10/2026-14:52:51] [TRT] [I] Engine generation completed in 290.4 seconds.
[03/10/2026-14:52:51] [TRT] [I] [MemUsageStats] Peak memory usage of TRT CPU/GPU
 memory allocators: CPU 0 MiB, GPU 12 MiB
TensorRT: export success ✅ 295.8s, saved as 'best.engine' (12.3 MB)

Export complete (296.6s)
Results saved to /home/jetson/ultralytics/runs/detect/train/weights
Predict:         yolo predict task=detect model=best.engine imgsz=640
Validate:        yolo val task=detect model=best.engine imgsz=640 data=orange.ya
ml
Visualize:       https://netron.app
💡 Learn more at https://docs.ultralytics.com/modes/export
jetson@ubuntu:~/ultralytics/runs/detect/train/weights$ ls
best.engine  best.onnx  best.pt  last.pt
jetson@ubuntu:~/ultralytics/runs/detect/train/weights$
```

References

https://docs.ultralytics.com/modes/train/